МІНІСТЕРСТВО ОСВІТИ І НАУКИ

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»



ЛАБОРАТОРНА РОБОТА № 7

З ДИСЦИПЛІНИ «ПРОГРАМНО-АПАРАТНЕ ЗАБЕЗПЕЧЕННЯ СИСТЕМ РУХОМИХ ОБ'ЄКТІВ»

ВИКОНАВ:

ВИШНЕВСЬКИЙ А. Г. (ІР-33)

ПРИЙНЯВ:

ШИХМАТ А.О

ЛЬВІВ 2026

НАЗВА:

Інтеграція телеметрії та протоколу MAVLink. Налаштування і перевірка обміну даними між безпілотною системою та наземною станцією керування.

МЕТА:

Набуття практичних навичок розгортання програмного середовища для моделювання безпілотної авіаційної системи, налаштування обміну MAVLink між ArduPilot SITL і наземною станцією QGroundControl, виконання автоматичної місії та збереження й аналізу телеметричних даних польоту.

ЗАВДАННЯ:

Варіант 5 (таблиця 7.6 методички). Номер варіанта для групи IP-33 дорівнює порядковому номеру студента в журналі. Параметри індивідуального завдання наведено в таблиці 1.

Таблиця 1 – Параметри індивідуального завдання (варіант 5)

Параметр	**Позначення**	**Одиниця**	**Значення**
Висота зльоту	H	м	50
Горизонтальна швидкість	V	м/с	5
Цільова точка, широта	latitude	градуси WGS 84	-35.3631166
Цільова точка, довгота	longitude	градуси WGS 84	149.1639735
Параметр ArduCopter WPNAV_SPEED	100 · V	см/с	500

Необхідно розгорнути Gazebo Sim, ArduPilot SITL і QGroundControl, налаштувати збереження журналів телеметрії (.tlog і .csv), створити в QGroundControl місію Takeoff – Waypoint – Land, виконати її в автоматичному режимі, після посадки проаналізувати CSV-файл, заповнити таблицю показників польоту, побудувати три графіки (висота, повітряна і наземна швидкості від часу) і пояснити призначення щонайменше трьох MAVLink-повідомлень.

РЕАЛІЗАЦІЯ:

1. Середовище та програмне забезпечення

Gazebo Sim, ArduPilot SITL і QGroundControl працюють у віртуальній машині Ubuntu 22.04 aarch64, створеній у UTM на Mac з процесором Apple Silicon. Така архітектура дозволяє запускати все програмне забезпечення без емуляції набору інструкцій. Для QGroundControl використано офіційну збірку для Linux aarch64; вікно програми відображається на віртуальному дисплеї Xvfb і транслюється по VNC. Версії всіх компонентів наведено в таблиці 2.

Таблиця 2 – Операційна система та програмне забезпечення

Компонент	Версія / параметри
Хост	MacBook Air, Apple M2 (arm64), 16 ГБ RAM
ОС хоста	macOS 27.0.1, архітектура arm64
Віртуалізація	UTM 4.7.5; віртуальна машина: 4 ядра, 6 ГБ RAM
ОС у віртуальній машині (тут працюють усі компоненти)	Ubuntu 22.04.5 LTS, ядро 5.15.0-198-generic, архітектура aarch64
Gazebo Sim	8.15.0 (Harmonic), світ iris_runway зі складу ardupilot_gazebo (коміт cc0290d)
ArduPilot SITL	ArduCopter V4.8.0-dev, коміт 665c0dee (від 18.09.2026)
Міст Gazebo – SITL	плагін ArduPilotPlugin (ardupilot_gazebo, коміт cc0290d), бекенд SITL --model json
MAVProxy	1.8.74
QGroundControl	v5.1.5 64 bit, QGroundControl-aarch64.AppImage, SHA-256 ddd92fe3…2c91e03
Бібліотеки для запуску QGC	libc6 2.39, libstdc++6 14.2.0, libgcc-s1 14.2.0 з Ubuntu 24.04 (3 пакети .deb)
Допоміжне ПЗ	Python 3.10.12, pymavlink 2.4.49 (у VM); Python 3.9, matplotlib 3.9.4 (аналіз на Mac)

Сумісність версій перевірено практично: QGroundControl v5.1.5 отримує HEARTBEAT від ArduCopter V4.8.0-dev, визначає тип апарата «мультиротор», отримує параметри і передає план місії; Gazebo Harmonic працює з плагіном ardupilot_gazebo через JSON-інтерфейс SITL.

2. Розгортання середовища (завдання 1)

Спосіб розгортання обрано самостійно: локальне встановлення у віртуальній машині без Docker. Gazebo Harmonic, ArduPilot SITL і робочий простір ros2_ws розгорнуто раніше для лабораторної №2 за офіційними інструкціями, тому тут повторно використано бінарник arducopter, плагін ardupilot_gazebo і набір світів. ROS 2 у цій роботі не використовується. Нове в цій роботі – наземна станція QGroundControl і скрипти швидкого запуску.

Завантаження QGroundControl і одноразова підготовка:

```
vishnevskiy@vishnevskiy:~$ mkdir -p ~/lab7/qgc && cd ~/lab7/qgc
vishnevskiy@vishnevskiy:~/lab7/qgc$ curl -L -C - --retry 20 -o QGroundControl-aarch64.AppImage \
    https://github.com/mavlink/qgroundcontrol/releases/download/v5.1.5/QGroundControl-aarch64.AppImage
vishnevskiy@vishnevskiy:~/lab7/qgc$ ~/lab7/scripts/setup_qgc.sh
```

Особливість: QGroundControl v5.1.5 зібрано під Ubuntu 24.04 і він вимагає glibc не нижче 2.38 та libstdc++ з GLIBCXX_3.4.32, тоді як у VM Ubuntu 22.04 (glibc 2.35). Системні бібліотеки без sudo не оновити, тому три пакети з архіву Ubuntu 24.04 розпаковано в окремий каталог без встановлення, а QGroundControl запускається через їхній завантажувач ld-linux (див. setup_qgc.sh). Решта програм у системі використовує рідні

бібліотеки. Вікно QGroundControl відображається на віртуальному дисплеї Xvfb і транслюється по VNC.

Ще одне рішення для швидкості симуляції: у VM немає GPU-прискорення, тому камера гімбала стандартної моделі iris_with_gimbal рендериться на CPU і знижує темп симуляції приблизно до 0,4–0,5 від реального часу (виміряно за відношенням sim_time до real_time у Gazebo). Файл CSV QGroundControl пишеться за годинником реального часу, тому такий темп спотворив би часову вісь. Скрипт start_sim.sh за замовчуванням підставляє у світ iris_runway.sdf модель iris_with_ardupilot (той самий дрон без гімбала і камери); смуга, рельєф, фізика, стартова точка і керування не змінюються. Виміряний темп симуляції в цьому режимі дорівнює 1,00 від реального. Режим зі стандартною моделлю лишається доступним: start_sim.sh gimbal.

Скрипти швидкого запуску

Створено набір bash-скриптів і допоміжних програм, які запускають усі компоненти без ручного введення команд. Призначення файлів наведено в таблиці 3, повні тексти основних скриптів – нижче.

Таблиця 3 – Призначення скриптів і файлів

Файл	**Призначення**
variant.env	Параметри варіанта (H, V, ціль) і перерахунок WPNAV_SPEED = 100 · V.
scripts/env.sh	Спільне середовище: ROS 2 Humble, Gazebo Harmonic, шляхи до ардупілотних пакетів, каталоги журналів.
scripts/start_sim.sh	Запускає Gazebo-сервер зі світом runway, ArduPilot SITL і MAVProxy; чекає готовності автопілота.
scripts/wait_ready.py	Через pymavlink слухає службовий порт 14551 і чекає біт PREARM_CHECK у SYS_STATUS.
scripts/setup_qgc.sh	Розпаковує AppImage QGroundControl і перевіряє залежності, друкує SHA-256.
scripts/start_qgc.sh	Запускає Xvfb :97, QGroundControl і VNC на порту 5901.
scripts/stop_sim.sh, stop_qgc.sh	Зупиняють усі процеси лаби й чистять залишки Fast DDS.
scripts/start_gui.sh, stop_gui.sh	Вікно Gazebo на дисплеї :98 з трансляцією по VNC, камера переходить до моделі дрона.
scripts/capture_frames.sh	Знімки віртуального екрана кожні 2 с для скриншотів початку й кінця польоту.
tools/make_plan.py	Формує план місії .plan за variant.env (Takeoff, Waypoint, Land).
tools/mav_recorder.py	Незалежний запис MAVLink (10 Гц) для перехресної перевірки CSV.
tools/analyze_tlog.py	Розбір .tlog: режими, arm/disarm, службові повідомлення, максимуми.
tools/analyze_csv.py	Рахує показники таблиці 7.4 за CSV QGroundControl і будує графіки.

Команди запуску і зупинки (виконуються у VM):

```
vishnevskiy@vishnevskiy:~$ ~/lab7/scripts/start_sim.sh
vishnevskiy@vishnevskiy:~$ ~/lab7/scripts/start_qgc.sh
vishnevskiy@vishnevskiy:~$ python3 ~/lab7/tools/make_plan.py ~/lab7/variant.env ~/lab7/variant5.plan
vishnevskiy@vishnevskiy:~$ ~/lab7/scripts/stop_qgc.sh
vishnevskiy@vishnevskiy:~$ ~/lab7/scripts/stop_sim.sh
```

start_sim.sh запускає три незалежні процеси, як у документації ArduPilot «SITL with Gazebo»: Gazebo-сервер (gz sim -s, без вікна), arducopter (--model json) і MAVProxy. MAVProxy бере TCP-порт 5760 у SITL і роздає MAVLink по UDP на 127.0.0.1:14550 (туди слухає QGroundControl, автопідключення UDP за замовчуванням) та 127.0.0.1:14551 (службовий порт для скриптів). Скрипт завершується, коли автопілот пройшов передпольотні перевірки (біт PREARM_CHECK у SYS_STATUS), це видно за рядком «Готово».

variant.env (параметри варіанта)

```
VARIANT=5

ALT_M=50
SPEED_MS=5
TARGET_LAT=-35.3631166
TARGET_LON=149.1639735

WPNAV_SPEED_CMS=$((SPEED_MS * 100))
WP_SPD_MS=$SPEED_MS
```

env.sh

```
#!/bin/bash

unset AMENT_PREFIX_PATH COLCON_PREFIX_PATH CMAKE_PREFIX_PATH PYTHONPATH LD_LIBRARY_PATH \
    GZ_SIM_RESOURCE_PATH GZ_SIM_SYSTEM_PLUGIN_PATH SDF_PATH
source /opt/ros/humble/setup.bash
export GZ_VERSION=harmonic
source "$HOME/ros2_ws/install/setup.bash"
export GZ_SIM_RESOURCE_PATH="$GZ_SIM_RESOURCE_PATH:$HOME/ros2_ws/install/ardupilot_gazebo/share"
export PATH="$PATH:$HOME/.local/bin"

export LAB7_DIR="$HOME/lab7"
export LAB7_RUN="$LAB7_DIR/run"
export LAB7_LOG="$LAB7_DIR/logs"
export LAB7_QGC="$LAB7_DIR/qgc"
export LAB7_DATA="$LAB7_DIR/logs/qgc_data"
mkdir -p "$LAB7_RUN" "$LAB7_LOG" "$LAB7_DATA"

export LAB7_DISPLAY=":97"
export LAB7_VNC_PORT=5901

source "$LAB7_DIR/variant.env"
```

start_sim.sh

```
#!/bin/bash

DIR="$(cd "$(dirname "$0")" && pwd)"
source "$DIR/env.sh"

MODE="${1:-nocam}"

"$DIR/stop_sim.sh" quiet

SITL_DIR="$LAB7_RUN/sitl"
mkdir -p "$SITL_DIR"
cd "$SITL_DIR" || exit 1

GZ_SHARE="$HOME/ros2_ws/install/ardupilot_gazebo/share/ardupilot_gazebo"
SITL_PARMS="$HOME/ros2_ws/install/ardupilot_sitl/share/ardupilot_sitl/config/default_params"
WORLD="$GZ_SHARE/worlds/iris_runway.sdf"
DEFAULTS="$SITL_PARMS/copter.parm"

if [ "$MODE" = "nocam" ]; then
    mkdir -p "$LAB7_DIR/worlds"
    sed 's#model://iris_with_gimbal#model://iris_with_ardupilot#' "$WORLD" > "$LAB7_DIR/worlds/iris_runway_nocam.sdf"
    WORLD="$LAB7_DIR/worlds/iris_runway_nocam.sdf"
    DEFAULTS="$DEFAULTS,$SITL_PARMS/gazebo-iris.parm"
else
    DEFAULTS="$DEFAULTS,$GZ_SHARE/config/gazebo-iris-gimbal.parm"
fi
DEFAULTS="$DEFAULTS,$LAB7_DIR/config/sitl_lab7.parm"

setsid gz sim -v3 -s -r "$WORLD" > "$LAB7_LOG/gazebo.log" 2>&1 < /dev/null &
echo $! > "$LAB7_RUN/gz.pid"

setsid bash -c 'arducopter --model json --speedup 1 --slave 0 --sim-address=127.0.0.1 --instance 0 --sysid 1 \
    --defaults "$0"; echo "SITL завершився з кодом $?"' "$DEFAULTS" > "$LAB7_LOG/sitl.log" 2>&1 < /dev/null &
echo $! > "$LAB7_RUN/sitl.pid"

setsid mavproxy.py --master tcp:127.0.0.1:5760 --sitl 127.0.0.1:5501 \
    --out 127.0.0.1:14550 --out 127.0.0.1:14551 --non-interactive \
    > "$LAB7_LOG/mavproxy.log" 2>&1 < /dev/null &
echo $! > "$LAB7_RUN/mavproxy.pid"
echo "Запущено (режим $MODE), журнали: $LAB7_LOG/{gazebo,sitl,mavproxy}.log. Очікування готовності..."

if python3 "$DIR/wait_ready.py" 180; then
    exit 0
fi
echo "ПОМИЛКА: система не стала готовою за 180 с, див. журнали в $LAB7_LOG"
exit 1
```

start_qgc.sh

```
#!/bin/bash
DIR="$(cd "$(dirname "$0")" && pwd)"
source "$DIR/env.sh"

SIZE="${1:-1600x900}"
"$DIR/stop_qgc.sh"

Xvfb "$LAB7_DISPLAY" -screen 0 "${SIZE}x24" > "$LAB7_LOG/xvfb.log" 2>&1 &
sleep 2
export DISPLAY="$LAB7_DISPLAY"
export LIBGL_ALWAYS_SOFTWARE=1
export QT_QUICK_BACKEND=software

cd "$LAB7_QGC/squashfs-root" || exit 1
setsid ./AppRun_glibc239 > "$LAB7_LOG/qgc.log" 2>&1 < /dev/null &
echo $! > "$LAB7_RUN/qgc.pid"

for i in $(seq 1 90); do
    WIN=$(xdotool search --onlyvisible --class "QGroundControl" 2>/dev/null | head -1)
    [ -n "$WIN" ] && break
    sleep 2
done
if [ -z "$WIN" ]; then
    echo "ПОМИЛКА: вікно QGroundControl не з'явилося, див. $LAB7_LOG/qgc.log"
    exit 1
fi
xdotool windowmove "$WIN" 0 0 windowsize "$WIN" "${SIZE%x*}" "${SIZE#*x}"

x11vnc -display "$LAB7_DISPLAY" -rfbauth "$HOME/.vnc/passwd" -allow 192.168.64.1 \
    -forever -shared -rfbport "$LAB7_VNC_PORT" -quiet > "$LAB7_LOG/vnc.log" 2>&1 &
sleep 1
echo "QGroundControl запущено (вікно $WIN). VNC: vnc://$(hostname -I | awk '{print $1}'):$LAB7_VNC_PORT"
```

setup_qgc.sh

```
#!/bin/bash
DIR="$(cd "$(dirname "$0")" && pwd)"
source "$DIR/env.sh"

cd "$LAB7_QGC" || exit 1
APPIMAGE=QGroundControl-aarch64.AppImage
[ -f "$APPIMAGE" ] || { echo "Немає $LAB7_QGC/$APPIMAGE"; exit 1; }
chmod +x "$APPIMAGE"

if [ ! -x squashfs-root/AppRun ]; then
    ./"$APPIMAGE" --appimage-extract > "$LAB7_LOG/qgc_extract.log" 2>&1 || { echo "Не вдалося розпакувати AppImage"; exit 1; }
fi

GLIBC239="$LAB7_DIR/glibc239/usr/lib/aarch64-linux-gnu"
if [ ! -x "$GLIBC239/ld-linux-aarch64.so.1" ]; then
    mkdir -p "$LAB7_DIR/glibc239/debs" && cd "$LAB7_DIR/glibc239/debs" || exit 1
    BASE=http://ports.ubuntu.com/ubuntu-ports/pool/main
    for pkg in g/glibc/libc6_2.39-0ubuntu8.9_arm64.deb \
               "g/gcc-14/libstdc++6_14.2.0-4ubuntu2~24.04.1_arm64.deb" \
               "g/gcc-14/libgcc-s1_14.2.0-4ubuntu2~24.04.1_arm64.deb"; do
        curl -sSL --retry 5 -O "$BASE/$pkg" || { echo "Не вдалося завантажити $pkg"; exit 1; }
    done
    for deb in *.deb; do dpkg -x "$deb" "$LAB7_DIR/glibc239"; done
    cd "$LAB7_QGC" || exit 1
fi

sed 's#^exec "\$QGC_BINARY" "\$@"#GLIBC239=""$GLIBC239""; LIBPATH="$GLIBC239:$LD_LIBRARY_PATH"; unset LD_LIBRARY_PATH LD_PRELOAD; exec "$GLIBC239/ld-
linux-aarch64.so.1" --library-path "$LIBPATH" "$QGC_BINARY" "$@"#' \
    squashfs-root/AppRun > squashfs-root/AppRun_glibc239
chmod +x squashfs-root/AppRun_glibc239
grep -c "ld-linux-aarch64.so.1" squashfs-root/AppRun_glibc239 > /dev/null || { echo "Не вдалося створити AppRun_glibc239"; exit 1; }

echo "SHA-256 AppImage: $(sha256sum "$APPIMAGE" | cut -d' ' -f1)"
echo "Готово. Запуск: squashfs-root/AppRun_glibc239"
```

wait_ready.py

```
import sys
import time

from pymavlink import mavutil

PREARM_CHECK = 0x10 000 000

timeout = float(sys.argv[1]) if len(sys.argv) > 1 else 180

link = mavutil.mavlink_connection("udpin:127.0.0.1:14 551")
start = time.time()

while time.time() - start < timeout:
    if link.wait_heartbeat(timeout=2):
        break
else:
    print("HEARTBEAT не отримано")
    sys.exit(1)
print(f"HEARTBEAT від системи {link.target_system} через {time.time() - start:.0f} с")

while time.time() - start < timeout:
    msg = link.recv_match(type="SYS_STATUS", blocking=True, timeout=2)
    if msg is None:
        continue
    if msg.onboard_control_sensors_health & PREARM_CHECK:
        print(f"Готово: дрон готовий до arm (через {time.time() - start:.0f} с).")
        sys.exit(0)

sys.exit(1)
```

config/sitl_lab7.parm (додаткові початкові параметри SITL)

```
DDS_ENABLE 0
```

Файл sitl_lab7.parm потрібен через збірку arducopter з ROS 2: DDS у ній увімкнено, і без агента micro-ROS процес завершується повідомленням «DDS: No ping response, exiting». Для роботи з MAVLink DDS не потрібен.

Перевірка: QGroundControl виявляє апарат

Після запуску start_sim.sh і start_qgc.sh QGroundControl автоматично встановив з'єднання: у верхній панелі з'явився стан «Ready», режим

Stabilize, 10 супутників GPS, заряд батареї 100 %, а на карті – апарат на аеродромі CMAC (рис. 1). Підключення визначається за повідомленнями HEARTBEAT апарата (тип 2 – квадрокоптер, автопілот 3 – ArduPilotMega).



Рисунок 1 – QGroundControl виявив апарат: стан «Ready», режим Stabilize, 10 супутників GPS

3. Налаштування QGroundControl і запису телеметрії (завдання 2)

У QGroundControl v5.1.5 розташування параметрів відрізняється від тексту методички: шлях збереження задається в Application Settings → General, а перемикачі журналів телеметрії лежать в окремому розділі Application Settings → Telemetry → Logging. Виконано такі дії:

- Application Load/Save Path змінено зі стандартного ~/Documents/QGroundControl на каталог лабораторної /home/vishnevskiy/lab7/logs/qgc_data (рис. 2). Відтепер там створюються підкаталоги Telemetry, Missions, Parameters та інші.
- У блоці Logging перемикач «Save log after each flight» уже був увімкнений за замовчуванням, а «Save CSV log of telemetry data» (запис усіх параметрів апарата в CSV з частотою 1 Гц під час польоту) вимкнений; його увімкнено (рис. 3).

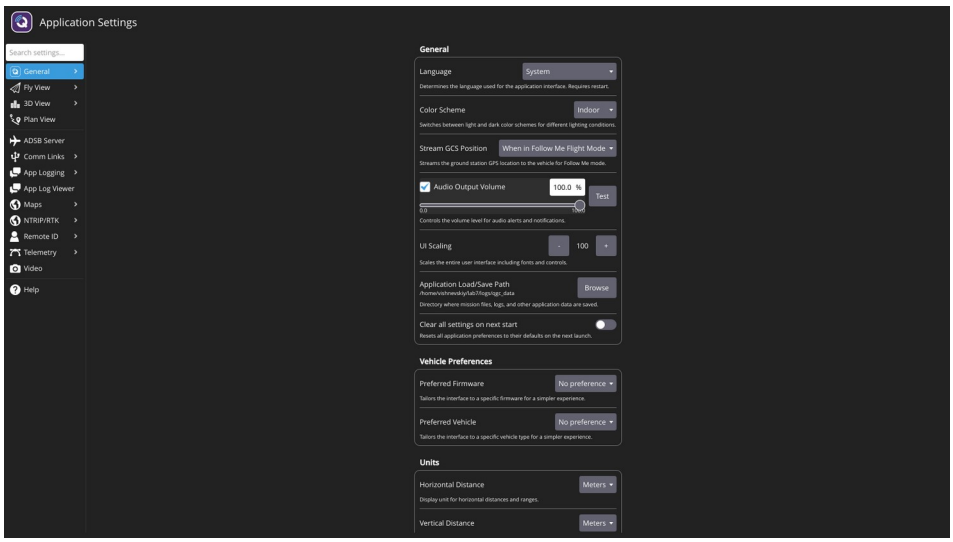


Рисунок 2 – Application Settings → General: Application Load/Save Path = /home/vishnevskiy/lab7/logs/qgc_data

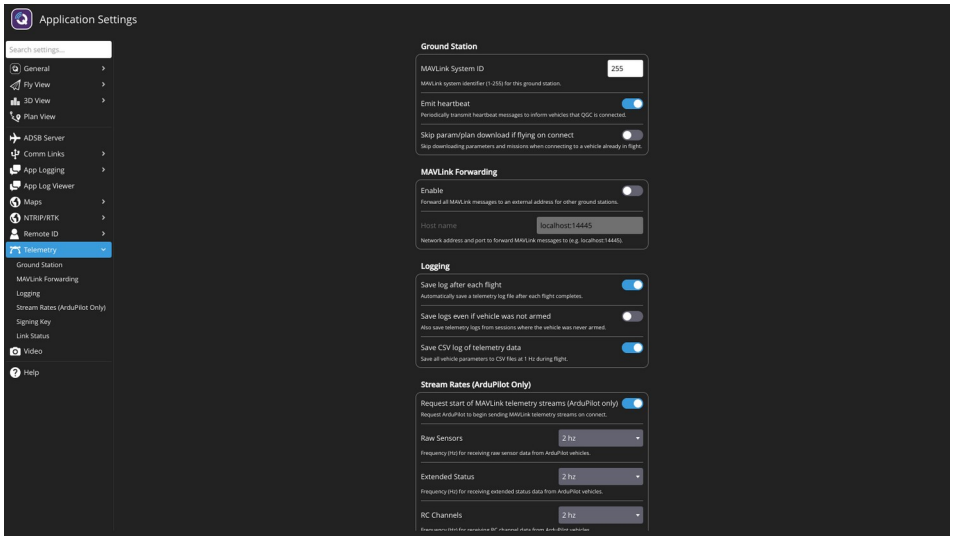


Рисунок 3 – Application Settings → Telemetry → Logging: увімкнено Save log after each flight і Save CSV log

Далі відкрито Analyze Tools → MAVLink Inspector. Усі потрібні повідомлення надходять від автопілота з частотою близько 4 Гц (HEARTBEAT – 1 Гц): HEARTBEAT (рис. 4), GLOBAL_POSITION_INT (рис. 5), ATTITUDE (у списку зліва) та SYS_STATUS (рис. 6). У списку також видно GPS_RAW_INT, VFR_HUD, HOME_POSITION, MISSION_CURRENT та інші.

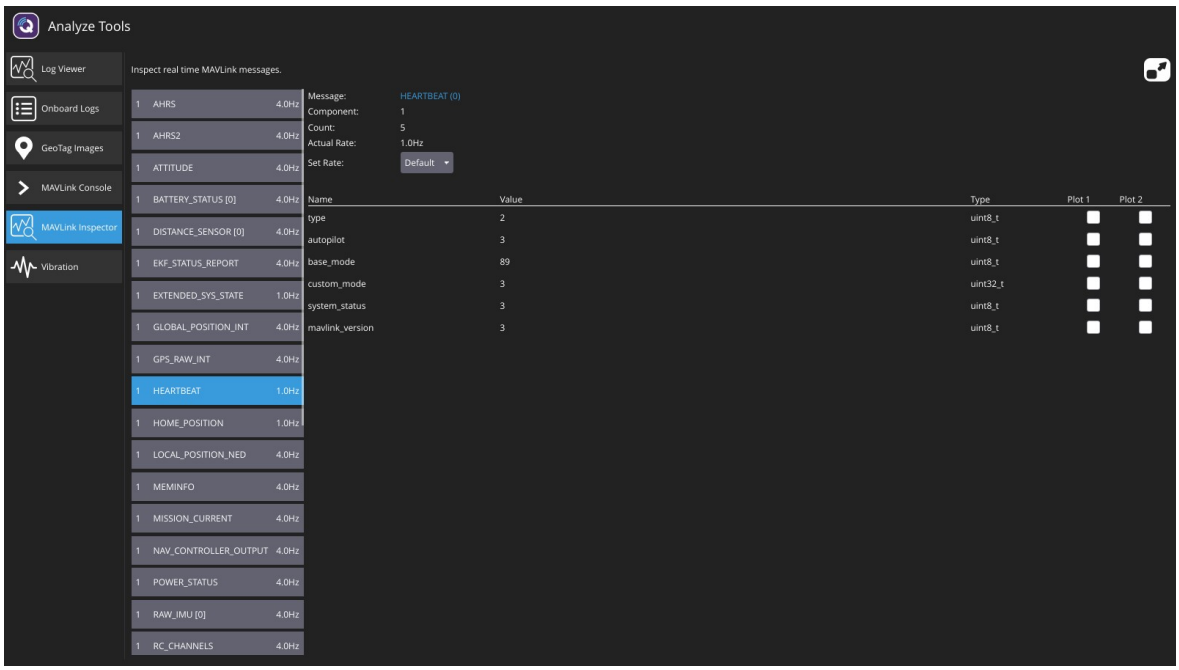


Рисунок 4 – MAVLink Inspector: HEARTBEAT (type = 2, autopilot = 3, custom_mode = 3, base_mode = 89)

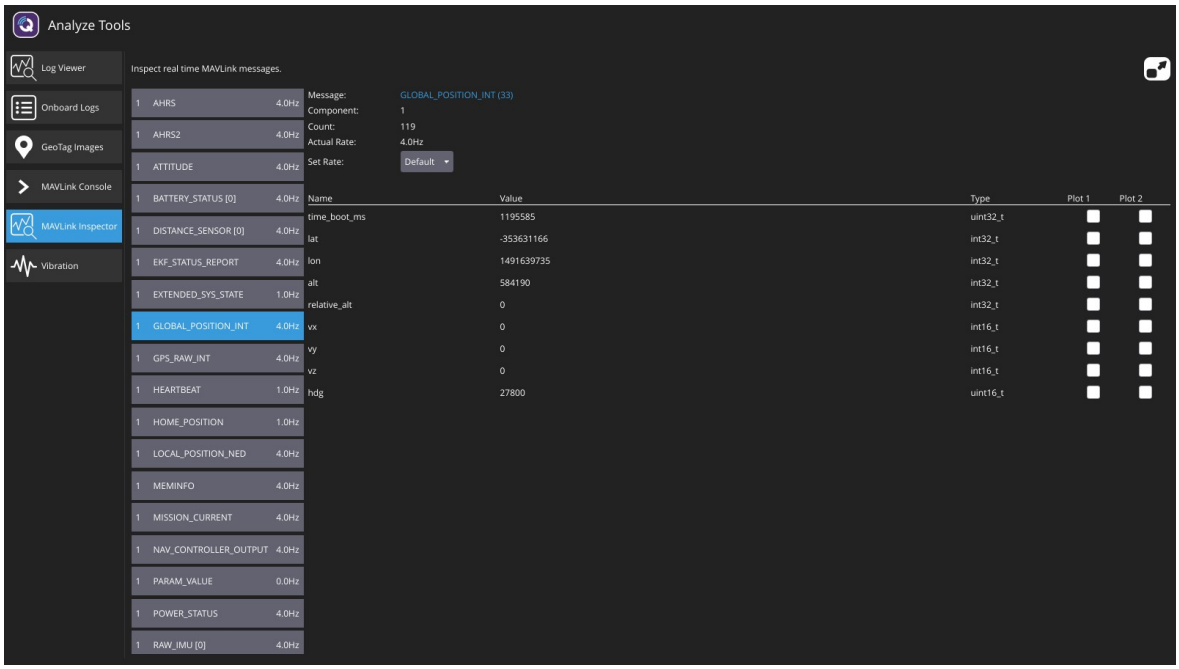


Рисунок 5 – MAVLink Inspector: GLOBAL_POSITION_INT (після посадки lat = -353631166, lon = 1491639735)

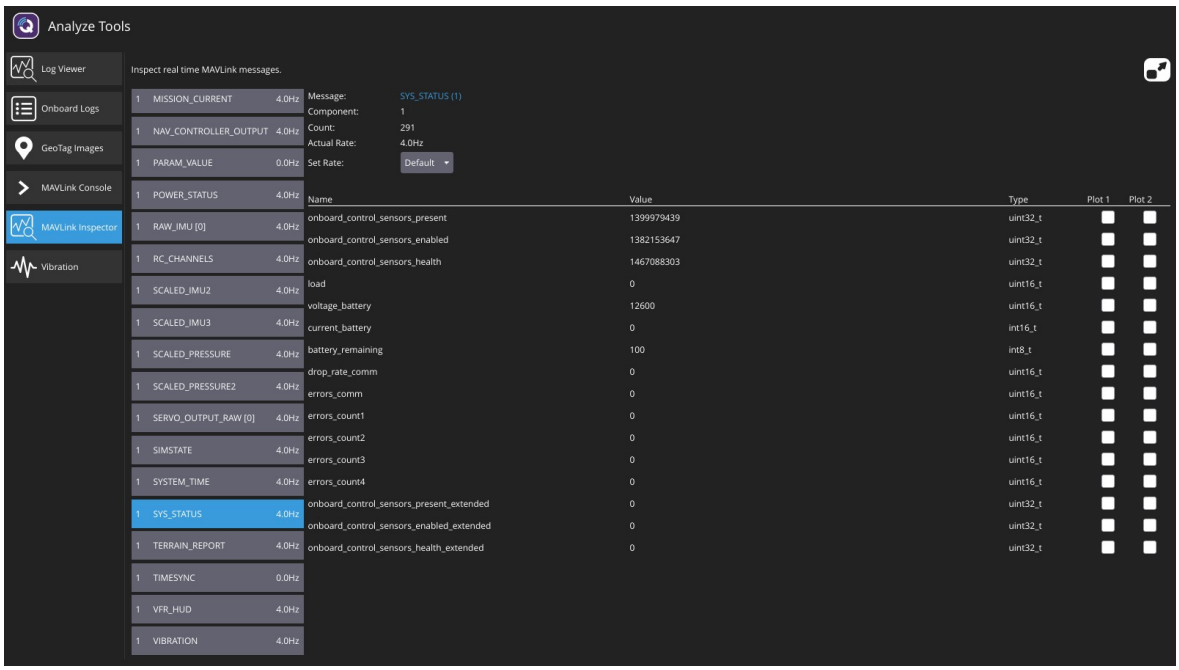


Рисунок 6 – MAVLink Inspector: SYS_STATUS (voltage_battery = 12600 мВ, battery_remaining = 100 %)

Призначення основних повідомлень MAVLink, які надходять від автопілота, наведено в таблиці 4.

Таблиця 4 – Призначення MAVLink-повідомлень, що спостерігалися в Inspector

Повідомлення	**Призначення**	**Що показує QGroundControl**
HEARTBEAT	Періодичний сигнал присутності апарата (1 Гц). Містить тип апарата, тип автопілота, режим польоту (custom_mode) і прапорець arm у base_mode. За його появою QGC вирішує, що з'єднання встановлено.	Тип «мультиротор», режим (Stabilize, Guided, Auto, Land), стан Armed/Disarmed.
GLOBAL_POSITION_INT	Позиція, оцінена автопілотом: широта і довгота (градуси · 10^7), висота над рівнем моря й відносно точки старту (мм), складові швидкості vx, vy, vz (см/с), курс.	Положення на карті, відносна висота, наземна швидкість.
ATTITUDE	Кути орієнтації апарата roll, pitch, yaw (рад) і кутові швидкості.	Штучний горизонт, значення крену й тангажа; з нього отримано максимальний roll.
SYS_STATUS	Стан бортових систем: які датчики присутні, ввімкнені й справні, напруга (мВ), струм (сА), залишок заряду батареї (%). Біт PREARM_CHECK показує, що автопілот пройшов передпольотні перевірки.	Індикатор батареї й повідомлення про готовність до arm.
GPS_RAW_INT	Сирі дані GNSS-приймача: тип фіксації (fix_type), кількість видимих супутників, координати, HDOP.	Індикатор GPS: тип fix і кількість супутників.

VFR_HUD	Дані для індикаторів пілота: повітряна швидкість (airspeed), наземна швидкість (groundspeed), висота, швидкість підйому, курс і газ.	Швидкості та висота в панелі приладів; джерело графіків повітряної і наземної швидкості.

4. Підготовка та виконання місії (завдання 3)

Параметр швидкості. Методичка вимагає встановити WPNAV_SPEED = 100 · V = 500 см/с. Перевірка показала, що в цій збірці ArduCopter (V4.8.0-dev) такого параметра немає: у версіях 4.7 і новіших групу WPNAV_* перейменовано на WP_*, а швидкість задається параметром WP_SPD в метрах за секунду. На запит PARAM_REQUEST_READ параметри WPNAV_SPEED, WPNAV_ACCEL, WPNAV_SPEED_UP, WPNAV_RADIUS автопілот не відповідає, а WP_SPD, WP_ACC, WP_SPD_UP, WP_RADIUS_M відповідають. Еквівалент заданого значення: 500 см/с = 5 м/с, тому встановлено WP_SPD = 5.0 м/с (було 10.0 м/с). Значення змінено у Vehicle Setup → Parameters (рис. 7, 8) і підтверджено незалежним читанням з автопілота.

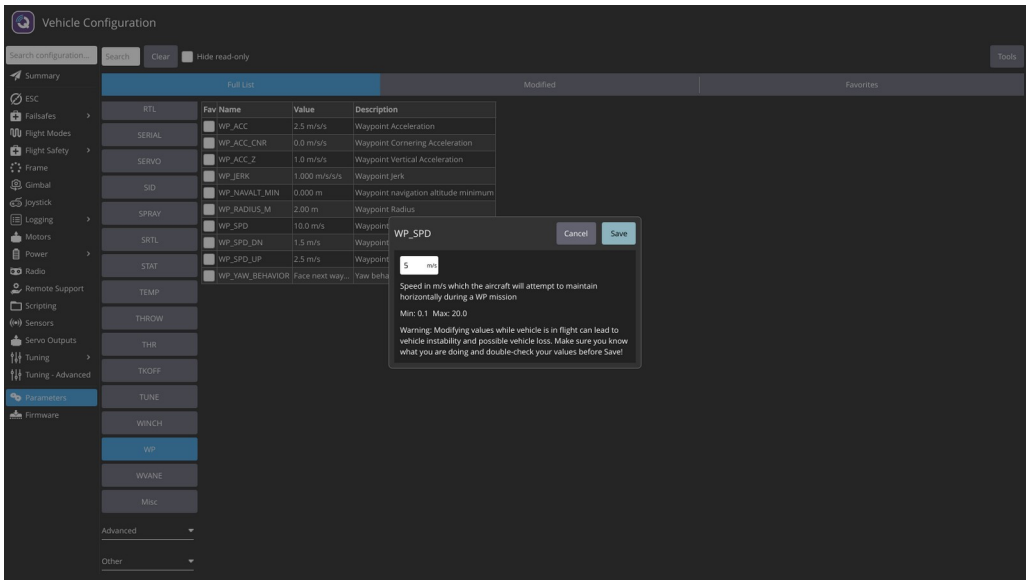


Рисунок 7 – Редагування параметра WP_SPD: введено значення 5 м/с

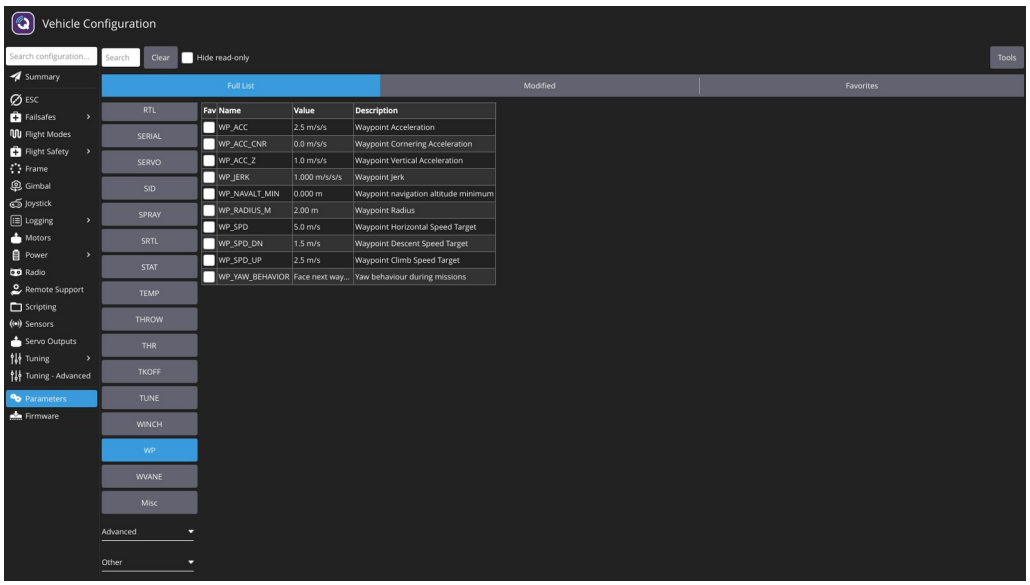


Рисунок 8 – Група WP після збереження: WP_SPD = 5.0 m/s

План місії складається з трьох елементів (таблиця 5). План сформовано скриптом make_plan.py за параметрами variant.env у форматі .plan, який QGroundControl відкриває на вкладці Plan так само, як і створений вручну (методичка допускає задання цілі окремим файлом плану місії QGroundControl). Висота Waypoint дорівнює висоті зльоту, Land виконується в цільовій точці.

Таблиця 5 – Створений план місії (варіант 5)

№	Елемент плану	Команда MAVLink	Параметри
1	Takeoff	NAV_TAKEOFF (22)	висота 50 м (відносно точки старту)
2	Waypoint	NAV_WAYPOINT (16)	широта -35.3631166, довгота 149.1639735, висота 50 м
3	Land	NAV_LAND (21)	широта -35.3631166, довгота 149.1639735

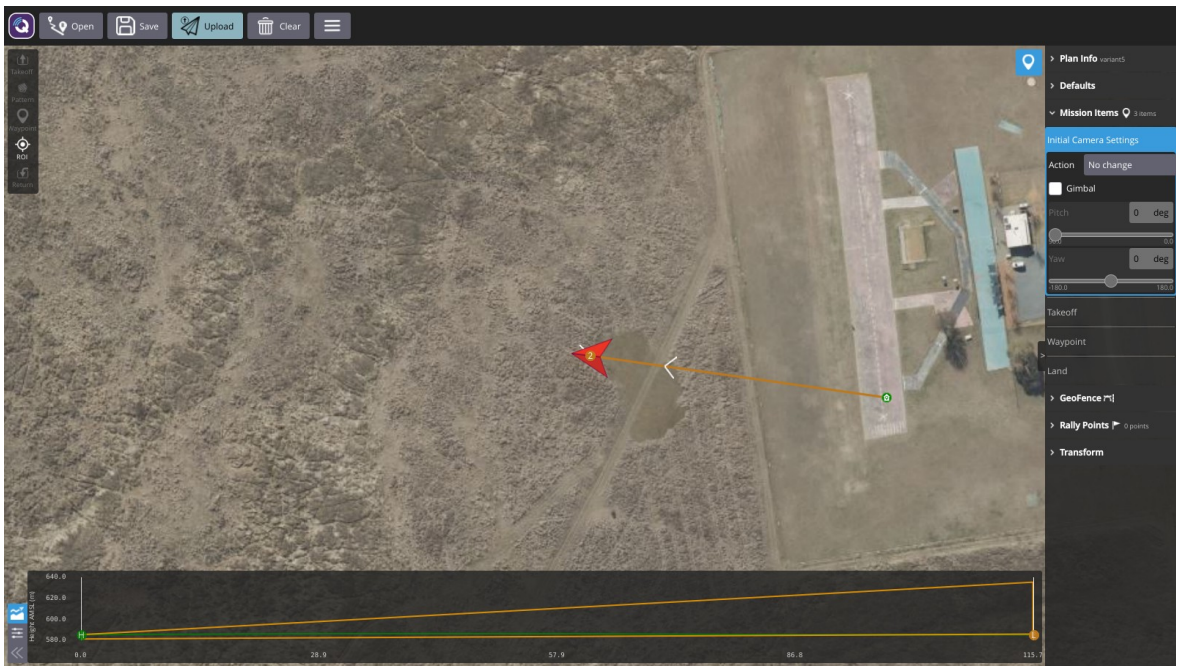


Рисунок 9 – Вкладка Plan: план variant5 із елементами Takeoff, Waypoint, Land; довжина маршруту 115,7 м

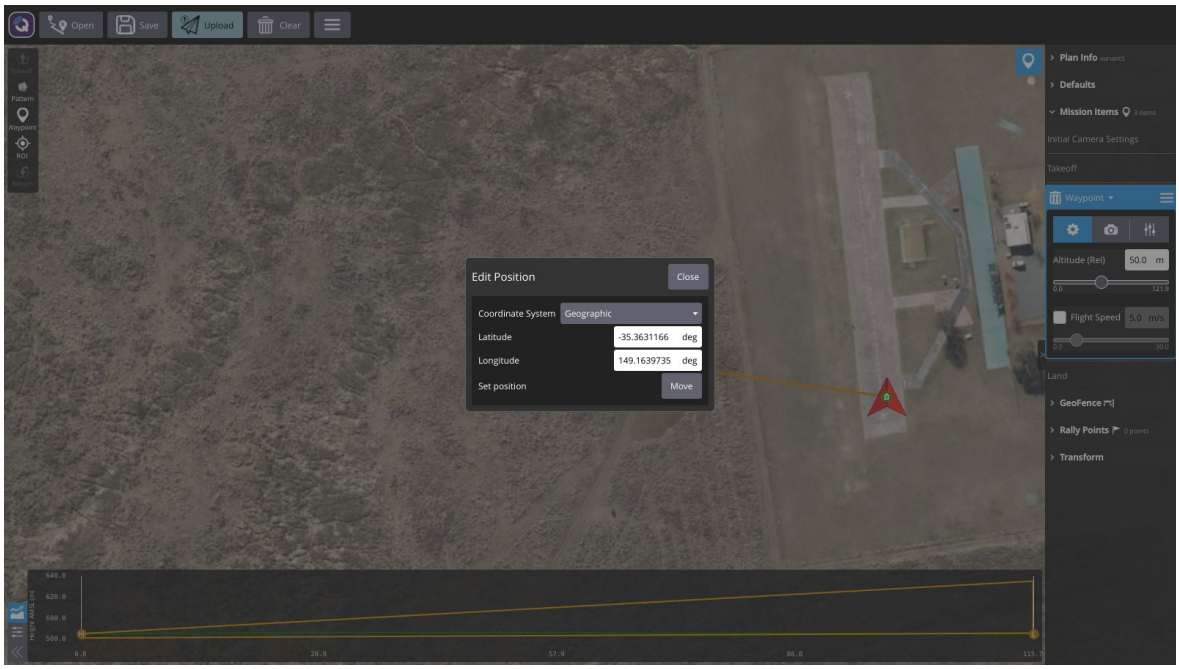


Рисунок 10 – Waypoint: Edit Position (Latitude -35.3631166, Longitude 149.1639735) і Altitude (Rel) 50.0 м

План передано в ArduPilot кнопкою Upload. Незалежна перевірка запитом MISSION_REQUEST_LIST: автопілот зберігає 4 елементи – точку home (seq 0) і три елементи плану з координатами й висотою з таблиці 5.

Виконання. На вкладці Fly arm виконано з панелі статусу «Ready» утриманням кнопки Arm (у QGroundControl 5.1 підтвердження дій робиться утриманням), одразу після цього – утриманням кнопки «Start Mission» (її підказка: «Takeoff and start the current mission»). За потоком

HEARTBEAT з .tlog QGroundControl виставив режим Guided, виконав arm і перевів апарат у Auto, після чого автопілот виконував елементи плану: Takeoff, Waypoint («Reached command #2»), Land. Початок місії показано на рис. 11, завершення польоту – на рис. 12, положення моделі в Gazebo після посадки – на рис. 13.



Рисунок 11 – Початок місії: «Flying», режим Auto, набір висоти (9,2 м, 2,5 м/с), таймер польоту 00:00:07

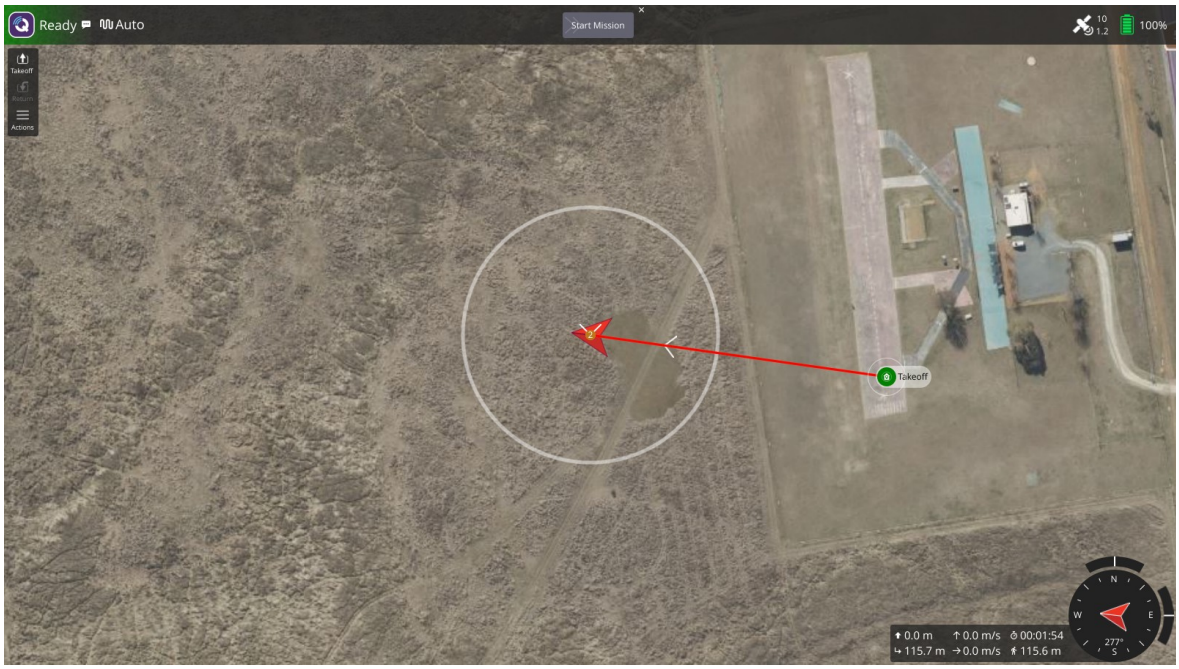


Рисунок 12 – Завершення польоту: «Ready», апарат у цільовій точці, висота 0,0 м, час польоту 00:01:54, 115,7 м від точки старту



Рисунок 13 – Gazebo Sim: модель iris_with_ardupilot у цільовій точці після посадки

Хронологія подій за .tlog (HEARTBEAT і STATUSTEXT апарата) наведена в таблиці 6.

Таблиця 6 – Хронологія польоту за .tlog

Час, UTC	Подія
21:41:36	перший arm (Stabilize); без команди на зліт автопілот сам роздармував апарат через DISARM_DELAY = 10 с (21:41:46)
21:41:59	режим Guided, arm; режим Auto; «Mission: 1 Takeoff»
21:42:25	завершено набір висоти 50 м, «Mission: 2 WP»
21:42:53	«Reached command #2» – досягнуто цільової точки, «Mission: 3 Land»
21:43:24	службове повідомлення «PrecLand: Failsafe Measures» (точна посадка не налаштована)
21:43:54	«Disarming motors» – апарат приземлився й роздармований у режимі Auto

Файли журналів у налаштованому каталозі /home/vishnevskiy/lab7/logs/qgc_data/Telemetry: «2026-10-21 21-50-01.tlog» (5,2 МБ, потік MAVLink усієї сесії з польотом) і «2026-10-21 21-41-37 vehicle1.csv» (CSV, 1 Гц, 196 стовпців). Обидва файли додаються до звіту. Зверніть увагу: QGroundControl записав .tlog у каталог під час наступного запуску програми (так він завершує тимчасовий файл сесії), а CSV дописувався до закриття програми, тому містить також рядки до і після польоту.

5. Аналіз телеметрії (завдання 4)

CSV відкрито і оброблено скриптом analyze_csv.py на Python (CSV відкривається у будь-якому табличному процесорі, але розрахунки й графіки зручніше відтворювати програмно). Спочатку виділено сам політ: від останнього скидання лічильника flightTime (початок останнього arm) до дотику до землі, тобто 114 рядків із 428. Етапи визначено за даними: зліт – до досягнення 99 % заданої висоти, горизонтальний політ – поки

наземна швидкість не менша за 0,5 м/с на заданій висоті, посадка – від зупинки над ціллю до дотику. CSV QGroundControl не містить режиму польоту й стану arm, тому ці два показники взято з .tlog (скрипт analyze_tlog.py). Результати зведено в таблицю 7.

Таблиця 7 – Результати аналізу телеметрії (за формою таблиці 7.4)

Показник	**Значення**
Час зльоту	arm і початок набору висоти о 21:41:59 UTC; набір заданих 50 м за 24 с
Найбільша висота	50.0 м (CSV); 50.02 м за GLOBAL_POSITION_INT з .tlog
Найбільша швидкість	5.0 м/с наземна, 5.0 м/с повітряна (середня на сталій ділянці 4.97 м/с)
Час польоту	114 с за лічильником QGC (arm → disarm); дотик до землі на 113 с; за .tlog arm → disarm 115.5 с
Початкова координата	-35.3632621; 149.1652374
Кінцева координата	-35.3631166; 149.1639735 (збігається із заданою)
Максимальний roll	5.0° за CSV (1 Гц); 5.29° за ATTITUDE з .tlog
Режими польоту	Stabilize → Guided → Auto (Takeoff, Waypoint і Land виконано в Auto)
Стан arm до та після місії	до: disarmed; під час польоту: armed (21:41:59 – 21:43:55 UTC); після: disarmed

Графіки висоти, повітряної і наземної швидкостей від часу (рис. 14–16) побудовано за CSV; етапи зльоту, горизонтального польоту і посадки виділено кольором, пунктиром показано задані значення.

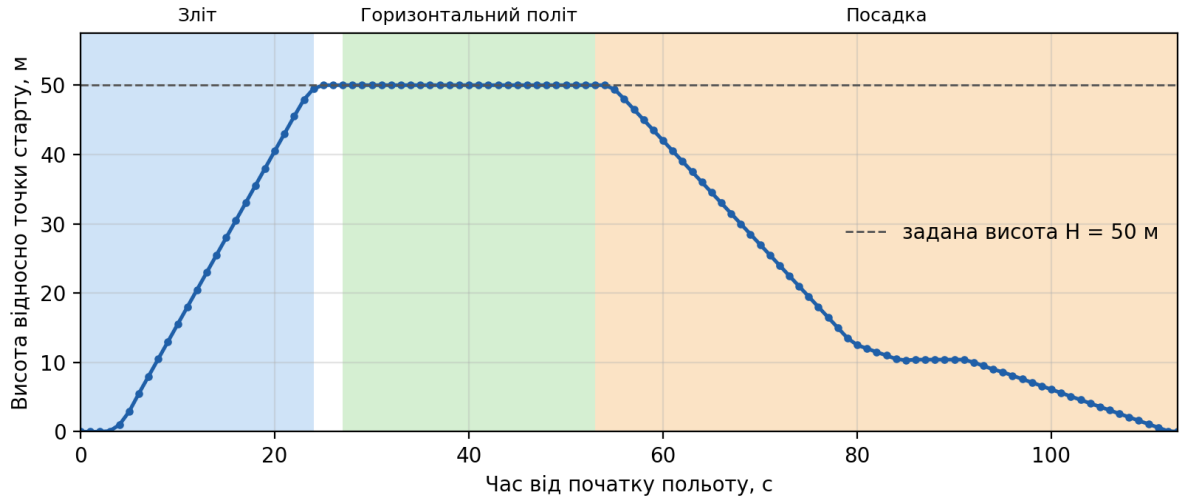


Рисунок 14 – Висота від часу

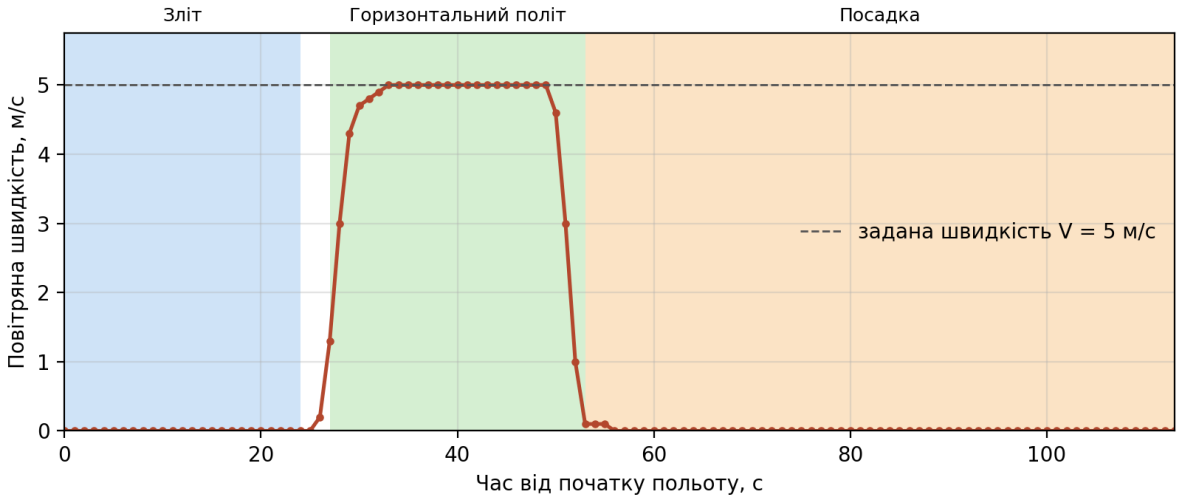


Рисунок 15 – Повітряна швидкість від часу

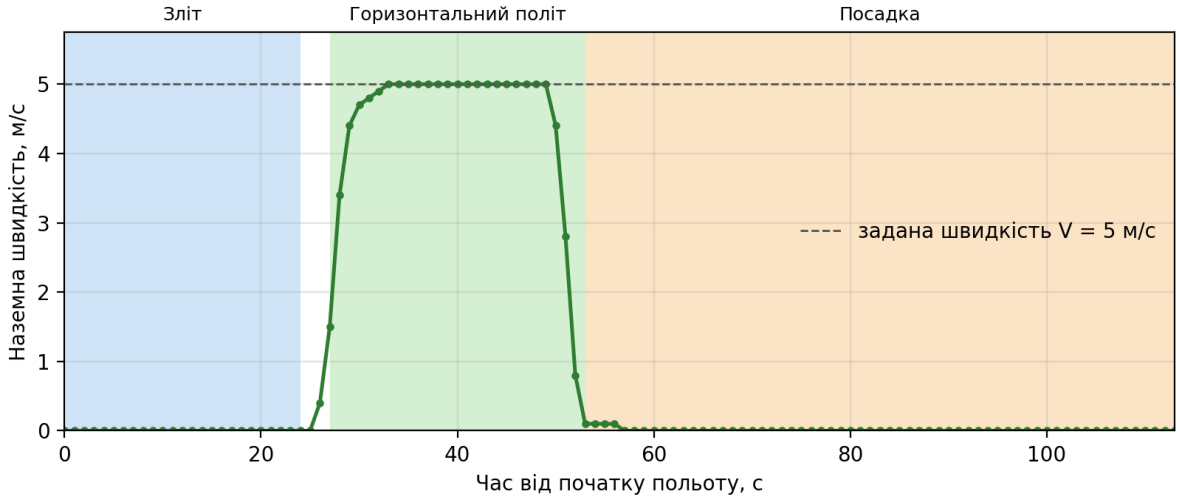


Рисунок 16 – Наземна швидкість від часу

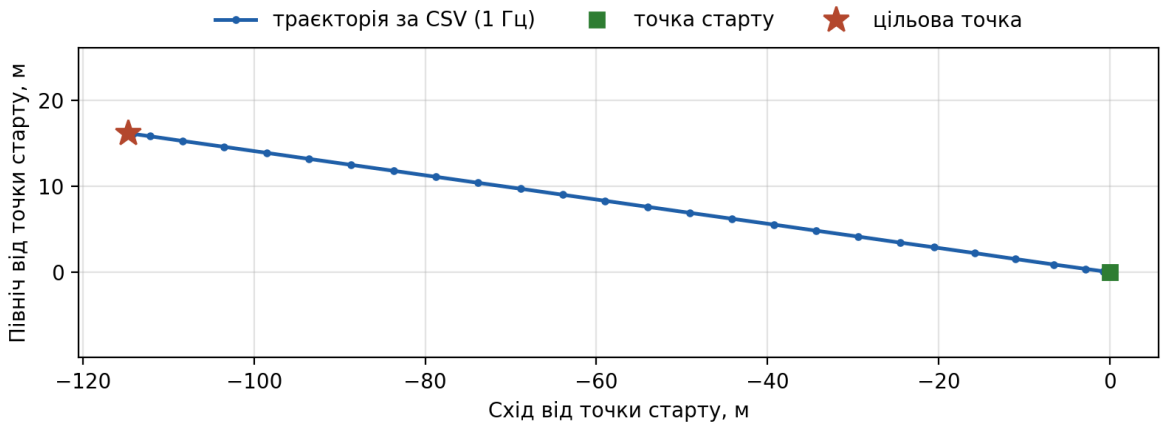


Рисунок 17 – Траєкторія польоту за CSV у локальних координатах відносно точки старту

Порівняння заданих параметрів із фактичними даними наведено в таблиці 8.

Таблиця 8 – Задані й фактичні параметри місії

Параметр	Задано	Фактично	Відхилення
Висота польоту H	50 м	50,0 м (CSV), 50,02 м (.tlog)	0,0…0,02 м
Горизонтальна швидкість V	5 м/с (WP_SPD = 5.0)	5,0 м/с, середня на сталій ділянці 4.97 м/с	розгін і гальмування
Цільова точка	-35.3631166; 149.1639735	-35.3631166; 149.1639735	< 0,1 м
Відстань від точки старту до цілі	–	115.7 м за QGC, пройдено 115.6 м	–

Пояснення відхилень і особливостей графіків:

- Швидкість. Найбільша наземна швидкість (5,0 м/с) збігається із заданою, але до неї дрон розганяється приблизно 7 с (на графіку швидкість зростає з 0 до 5 м/с з 26-ї до 33-ї секунди), а в кінці гальмує. Причина – обмеження прискорення автопілотом (WP_ACC = 2,5 м/с²); тому середня швидкість за весь горизонтальний етап менша за задану.
- Висота. Підйом триває з постійною швидкістю 2,5 м/с (WP_SPD_UP), завдяки чому 50 м досягається за 24 с. Незначне перевищення (0,02 м за .tlog) лежить у межах точності регулятора висоти.
- Посадка. Зниження виконується зі швидкістю 1,5 м/с (WP_SPD_DN) до висоти 10 м (LAND_ALT_LOW_M), де є площадка приблизно на 10 с, що збігається з повідомленням «PrecLand: Failsafe Measures»; потім швидкість зниження зменшується до 0,5 м/с (LAND_SPD_MS) аж до дотику.
- Цільова точка. Місія завершується посадкою в тих самих координатах, тому остання позиція збігається з заданою до сьомого знака після коми (< 0,1 м). У симуляції немає вітру (SIM_WIND_SPD = 0), а положення, які бачить автопілот, майже не мають шуму (кінцеві координати в CSV стабільні й збігаються до 7-го знака), що пояснює таку точність; реальний апарат дав би відхилення порядку метра.
- Повітряна швидкість. ArduCopter не має датчика повітряної швидкості, тому поле airSpeed є оцінкою автопілота; за відсутності вітру воно майже збігається з наземною швидкістю (максимальна різниця в CSV – 0,4 м/с під час розгону), тому графіки рис. 15 і 16 схожі.
- Roll. Найбільший крен 5,0° за CSV і 5,29° за ATTITUDE з .tlog (CSV фіксує лише одне значення за секунду, тому пік повного потоку трохи вищий); найбільший тангаж 13,3° припадає на розгін і гальмування.

Якість телеметрії. Потік MAVLink стабільний: частота повідомлень 4 Гц (HEARTBEAT – 1 Гц), без втрат (drop_rate_comm = 0, errors_comm = 0 у SYS_STATUS), GPS: тип фіксації 6, 10 супутників, HDOP 1,2. Обмеження CSV: частота 1 Гц, тому короткі процеси (розгін, гальмування) відображаються кількома точками; напруга батареї (12,6 В) і заряд (100 %) за весь політ не змінюються, бо модель батареї SITL не

розряджається. Для детального аналізу використано .tlog (повна частота ATTITUDE, GLOBAL_POSITION_INT).

6. Відхилення від методички та особливості виконання

- Параметр WPNAV_SPEED у ArduCopter V4.8.0-dev відсутній, його замінює WP_SPD (м/с): задано WP_SPD = 5 м/с, що відповідає WPNAV_SPEED = 500 см/с.
- У Gazebo використано модель iris_with_ardupilot (без камери гімбала) у стандартному світі iris_runway: світ і стартову точку не змінено, замінено лише модель дрона; причина – продуктивність без GPU (темп 1,00 замість ≈0,4–0,5).
- QGroundControl запущено у Linux-VM на віртуальному дисплеї, а не нативно на Mac: офіційна збірка aarch64 вимагає нових бібліотек, які підкладено окремо (setup_qgc.sh).
- Каталог журналів – /home/vishnevskiy/lab7/logs/qgc_data замість прикладу /logs (у прикладі потрібні права root).
- Перемикачі журналів у QGroundControl 5.1 знаходяться в Telemetry → Logging, а не в General.
- План місії створено скриптом у форматі .plan і відкрито в QGroundControl; Upload, перегляд і перевірка виконано в графічному інтерфейсі.
- CSV не містить режиму й стану arm, ці показники взято з .tlog; .tlog записано QGroundControl під час наступного запуску.
- Перший arm (21:41:36) автопілот скасував через 10 с, бо зліт не було запущено (DISARM_DELAY = 10 с); тому arm і Start Mission виконано послідовно без паузи. Окрема спроба «disarm» у польоті була відхилена автопілотом (повідомлення Arm/Disarm failed у QGroundControl), політ не переривався.
- Розрахунки й графіки виконано скриптом на Python замість табличного процесора.

Додатки

До звіту додаються файли телеметрії та результатів:

- 2026-10-21 21-50-01.tlog – журнал MAVLink сесії QGroundControl, збережений у каталозі Telemetry;
- 2026-10-21 21-41-37 vehicle1.csv – CSV телеметрії з частотою 1 Гц;
- variant5.plan – план місії у форматі QGroundControl;
- tlog_summary.json, metrics_csv.json – результати розбору .tlog і CSV;
- скрипти і конфігурації: репозиторій https://github.com/atexx-v/pazns, папка lab7.

ВИСНОВКИ:

У роботі розгорнуто відтворюване середовище Gazebo Sim, ArduPilot SITL і QGroundControl, налаштовано обмін MAVLink між автопілотом і наземною станцією та виконано автоматичну місію за варіантом 5: зліт на 50 м, політ до точки (-35.3631166; 149.1639735) зі швидкістю 5 м/с і посадка. Усі параметри місії виконано: найбільша висота 50,0 м, найбільша швидкість 5,0 м/с, кінцева координата збігається із цільовою, час польоту 114 с, апарат після посадки автоматично роздармований.

Практично перевірено, як працює MAVLink: за HEARTBEAT QGroundControl визначає наявність апарата, його тип і режим; GLOBAL_POSITION_INT, ATTITUDE, VFR_HUD і SYS_STATUS дають позицію, орієнтацію, швидкості й стан систем; наземна станція передає параметри (WP_SPD) і точки місії (MISSION_ITEM_INT), а автопілот підтверджує їх. Збережено .tlog (повний потік повідомлень) і CSV (1 Гц, зручний для таблиць), їх можливості доповнюють одна одну: у CSV немає режиму й arm, а .tlog не такий зручний для швидкого аналізу.

Телеметрія якісна й узгоджена із завданням, але має обмеження симуляції: немає вітру і шуму GPS, тому точність виходу в ціль нетипово висока, батарея не розряджається, а повітряна швидкість лише оцінюється. Окремі відхилення від методички (параметр WP_SPD замість WPNAV_SPEED, модель дрона без камери, запуск QGroundControl у VM) пояснені в розділі 6.